

RVT-Swarm: Recoverability-Verified Topological Control for Decentralized Swarm Formation

Anonymous Authors

Abstract—Decentralized swarm formation in cluttered environments requires more than instantaneous collision avoidance: the team must also preserve the ability to reorganize and recover a usable formation after passing through bottlenecks. We present RVT-Swarm, a graph-based controller that augments per-robot acceleration commands with discrete topology actions and a learned *recoverability* score map over candidate formations. Recoverability is defined as a short-horizon ability to remain collision-free, make goal progress, and return toward a formation tube under a candidate topology. RVT-Swarm combines (i) a local interaction-graph encoder with action, recoverability, uncertainty, and topology heads; (ii) a counterfactual topology selector with hysteresis and cooldown; and (iii) a lightweight CBF-inspired progress-preserving projection shield. Training uses imitation learning plus counterfactual short-horizon rollouts from a common expert controller. On the current prototype benchmark (400 evaluation episodes per method, single seed), InstantCert attains the highest success at 45.25%, while RVT-Swarm reaches 42.50% success, 49.0% formation satisfaction, and the lowest stall (2.82%) and deadlock (8.25%) rates among the learned methods. The ablation study reveals a sharper implementation-level conclusion: recoverability modeling, counterfactual topology selection, and the progress shield all improve over their removals, but a *no-topology* variant currently matches or exceeds the full model on end-task success. We therefore frame RVT-Swarm as an artifact-faithful research scaffold for recoverability-aware swarm formation control whose liveness benefits are already visible, but whose topology adaptation remains the least mature component.

Index Terms—Multi-robot systems, swarm formation control, graph neural networks, safety-critical control, topology adaptation, recoverability.

I. INTRODUCTION

Multi-robot formations are useful only when they remain both *safe* and *task-capable*. In warehouse logistics, cooperative inspection, or search-and-rescue, a team must not merely avoid collisions; it must also preserve enough structure to pass through clutter, regroup, and continue making progress toward a collective goal [1], [2], [3]. This is especially difficult in decentralized settings with limited local sensing, heterogeneous local contexts, and team sizes that vary at deployment time.

Existing approaches usually solve only part of this problem. Graph-based decentralized controllers scale well to large teams but often optimize local motion directly, without an explicit notion of whether the team can still *recover* a useful formation after local reconfiguration [4], [5]. Control-barrier-function (CBF) approaches provide strong instantaneous safety

structure [6], [7], [8], but they typically reason about collision avoidance rather than return to a formation tube. Adaptive-formation approaches can change shape to fit the environment, but usually rely on hand-designed switching logic or implicit policies without an explicit recoverability score over alternative topologies [2], [3]. Reactive methods such as ORCA can maintain goal progress, but often do so by sacrificing formation quality entirely [9].

This paper explores a different angle: rather than asking only whether the current action is safe, we ask whether the current local state is *recoverable* under a small set of candidate formation topologies. In RVT-Swarm, each local graph observation is mapped not only to a control action but also to a score map over topological actions (KEEP, COMPRESS, LINE, SPLIT, RECOVER). The controller then evaluates these topologies counterfactually and selects the one with the best predicted recoverability, subject to hysteresis and cooldown. A lightweight projection-based shield intervenes only when risk is high or when all candidate topologies appear unrecoverable.

The resulting system is deliberately pragmatic. It does not claim an exact viability kernel or a formal distributed reachability proof. Instead, it uses short-horizon counterfactual rollouts to construct a learned surrogate of recoverability and then deploys that surrogate online for topology adaptation. This design is consistent with the current codebase and the current benchmark setting.

A. Contributions

We make the following contributions:

- 1) We formulate *recoverability-aware topology adaptation* for decentralized swarm formation, where candidate formation modes are scored by their short-horizon ability to remain safe, make goal progress, and return toward a formation tube.
- 2) We implement a graph-attention controller with four coupled outputs: per-robot accelerations, graph-level topology logits, per-topology recoverability scores, and uncertainty estimates. A gradient-scaling mechanism reduces interference from auxiliary heads on the action pathway.
- 3) We introduce a recoverability-aware topology selector with hysteresis, cooldown, contextual bonuses, and a progress-preserving CBF-inspired projection shield that only activates under elevated risk or global negative recoverability.

- 4) We provide a unified experimental scaffold with learned baselines, historical baselines, rollout-based labels, matched-seed evaluation, and liveness metrics such as deadlock, topology switching, formation recovery time, and irreversible collapse. The released artifact exposes both the gains and the current failure modes of recoverability-aware topology control.

II. RELATED WORK

A. Decentralized Graph Policies

Graph neural networks are a natural fit for decentralized multi-robot control because they preserve permutation invariance and local communication structure [4], [5]. Our work follows this line in using a local interaction graph, but adds explicit topology adaptation and a recoverability score map rather than predicting control alone.

B. Safety-Critical Multi-Robot Control

Recent safety-critical multi-robot controllers use graph-structured certificates, collaborative safety filters, or discrete safe-policy learning to enforce pairwise constraints in distributed teams [6], [7], [8]. Recent graph-CBF work scales safety reasoning through local graphs [6], [8]. RVT-Swarm is influenced by this design, but our shield is intentionally lightweight and approximate: it is a projection-based correction mechanism rather than a formally certified QP solver. More importantly, our main decision variable is not only the continuous action but also the topology under which the swarm remains recoverable.

C. Adaptive Formation and Recoverability

Adaptive formation methods change the team geometry to fit corridors, obstacles, or task context [2], [3]. Viability and reachability learning study whether a state can remain in or return to a desirable set over a finite horizon [10], [11], [12]. RVT-Swarm sits between these directions: it learns a short-horizon, local-graph surrogate of recoverability and uses it to guide topology changes. The current implementation should therefore be understood as a recoverability-informed formation controller rather than as a formal reachability engine.

To make this positioning explicit, Table I contrasts RVT-Swarm with representative prior work along the axes that matter most for the current prototype: local graph execution, formation awareness, explicit topology adaptation, recoverability modeling, counterfactual selection, and progress-preserving safety intervention.

III. PROBLEM SETUP

A. Environment and Dynamics

We consider N robots moving in a 2D workspace with discrete-time kinematics

$$\mathbf{p}_i(t+1) = \mathbf{p}_i(t) + \mathbf{v}_i(t)\Delta t, \quad \mathbf{v}_i(t+1) = \mathbf{v}_i(t) + \mathbf{u}_i(t)\Delta t, \quad (1)$$

subject to bounded speed and acceleration. Each robot observes local relative geometry, obstacle context, goal direction, formation error, and a 36-ray 270° LiDAR scan. The

environment includes static obstacles, dynamic obstacles, and a narrow-passage configuration.

B. Formation Tube and Topology Actions

Let $\bar{\mathbf{p}}(t)$ denote the swarm centroid and let $\{\delta_i^\tau\}_{i=1}^N$ denote the desired offsets of topology $\tau \in \mathcal{T}$. The current formation error is

$$\mathbf{e}_i^\tau(t) = \bar{\mathbf{p}}(t) + \delta_i^\tau - \mathbf{p}_i(t). \quad (2)$$

Rather than choosing only a continuous control input, the policy also selects a discrete *formation mode* (or topology action) from a small finite set:

$$\mathcal{T} = \{\text{KEEP}, \text{COMPRESS}, \text{LINE}, \text{SPLIT}, \text{RECOVER}\}. \quad (3)$$

Intuitively, these modes tell the swarm whether to keep its current shape, compress into a tighter pattern, align into a line, temporarily split into subgroups, or recover back toward the nominal formation. These actions control the global formation geometry through scale, corridor alignment, or temporary splitting/merging. In the released simulator, topology actions act through four latent state variables: `topology_mode`, `formation_scale`, `split_active`, and `subteam_ids`. COMPRESS contracts the nominal spacing, LINE aligns desired offsets with the inferred corridor, SPLIT partitions robots into two laterally separated lanes, RECOVER expands the formation and clears split state, and KEEP either preserves the current corridor-aligned pattern inside a bottleneck or relaxes the swarm back toward the nominal grid outside the bottleneck.

C. Recoverability

For a state s_t and topology τ , we define recoverability as a short-horizon score that rewards collision-free motion, goal progress, and return toward a formation tube while penalizing deadlock, excessive switching, and persistent split states:

$$R(s_t; \tau) = \frac{1}{H} \sum_{h=0}^{H-1} \left[w_c c_{t+h} + w_p p_{t+h} + w_f f_{t+h} - w_d d_{t+h} - w_s q_{t+h} \right]. \quad (4)$$

Here c_{t+h} is a collision-free indicator, p_{t+h} measures goal progress, f_{t+h} measures formation-tube recovery, d_{t+h} is a deadlock-related penalty, and q_{t+h} captures switching/splitting penalties. In the implementation, these scores are generated by short-horizon counterfactual rollouts of a common expert controller. The released code instantiates this template with fixed coefficients: 0.95 for collision-free execution, 0.65 for goal progress, 1.55 for formation-tube score, and an additional recoverability-proxy term, together with topology-specific bonuses for RECOVER and bottleneck-aligned LINE/SPLIT, and penalties for deadlock, switch count, and lingering split states.

The online objective is then to select $(\mathbf{u}, \tau)$ that preserves progress while favoring positive recoverability:

$$\max_{\mathbf{u}, \tau} \alpha \text{progress} + \beta \text{formation} + \gamma \hat{R}(s_t; \tau), \quad (5)$$

where $\hat{R}$ is the learned recoverability surrogate.

TABLE I: Positioning of RVT-Swarm relative to representative recent prior work. ✓ = primary capability; △ = partially addressed; ✗ = not addressed. The table is intended as a qualitative positioning summary for the current prototype rather than an exhaustive taxonomy.

Method	Venue	Year	Local graph distributed	Formation aware	Obstacle dynamics	Topology adaptation	Recoverability model	Counterfactual selection	Progress shield
GCBF+ [6]	T-RO	2025	✓	△	✓	✗	✗	✗	△
Collab. safety form. [7]	arXiv	2024	✓	✓	△	✗	✗	✗	△
Neural barrier reach [10]	IFAC	2024	✗	✗	△	✗	✓	✗	✗
Cert. Reach. [11]	RA-L	2025	△	✗	✓	✗	✓	✗	✗
RAIL [12]	ICRA	2025	△	✗	✓	✗	✓	✗	✗
DGPPO [8]	ICLR	2025	✓	✗	✓	✗	✗	✗	✓
Form.-adapt nav. [2]	arXiv	2024	✓	✓	△	✓	✗	✗	✗
Multi-UAV RL [3]	IROS	2025	✓	✓	✓	△	✗	✗	✗
RVT-Swarm (ours)	—	2026	✓	✓	✓	✓	✓	✓	✓

IV. METHOD

RVT-Swarm combines a graph-attention backbone, a recoverability-aware multi-head decoder, a counterfactual topology selector, and a projection-based shield. Figure 1 shows the overall pipeline and Figure 2 details the recoverability network.

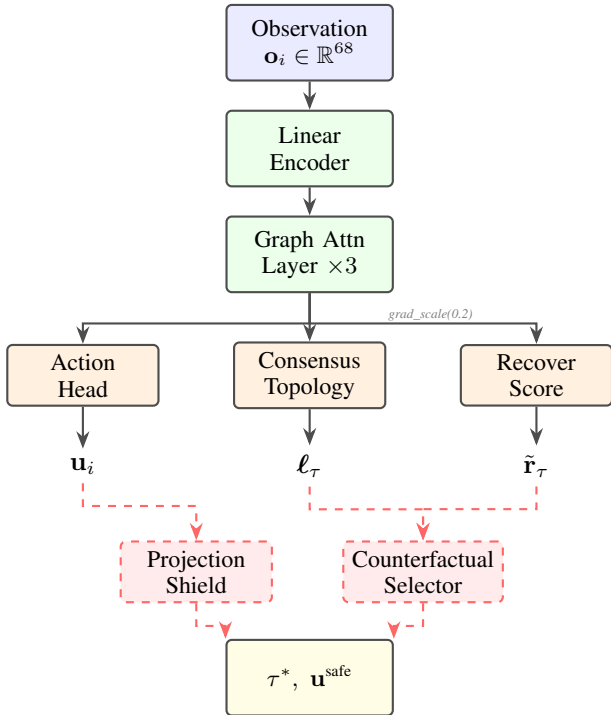


Fig. 1: Overview of RVT-Swarm. Local graph observations feed a shared graph-attention encoder whose outputs are decoded into control actions, topology logits, and per-topology recoverability scores. A counterfactual selector and a lightweight projection shield operate only at inference time.

A. Local Graph Encoder

At each step, the swarm builds a k -nearest-neighbor interaction graph with $k = 6$. Each node contains a 68-dimensional feature vector including position, velocity, heading, goal vector, centroid-relative position, formation error, obstacle-relative cues, topology encoding, normalized minimum TTC proxy,

and 36 normalized LiDAR ranges. Each edge contains 11-dimensional relational features, including relative position, relative velocity, spacing error, corridor projections, and a TTC-style safety descriptor.

More explicitly, the per-robot feature vector is

$$\mathbf{x}_i = [\mathbf{p}_i, \mathbf{v}_i, \text{head}_i, \mathbf{g}_i, \hat{\mathbf{g}}_i, (\mathbf{p}_i - \bar{\mathbf{p}}), \mathbf{e}_i^\tau, (\mathbf{p}_i - \mathbf{p}_{\text{obs},c}), \Pi_i^{\text{corr}}, \Pi_i^{\text{lat}}, \kappa_t, b_t, \pi_t, \rho_i^{\text{local}}, d_i^{\text{min}}, \mathbf{v}_i^{\text{dyn}}, \chi_t, \text{onehot}(m_t), \text{TTC}_i^{\text{min}}, \text{LiDAR}_{i,1:36}], \quad (6)$$

where $\text{head}_i = (\cos \theta_i, \sin \theta_i)$ is a velocity-derived heading encoding, $\mathbf{g}_i$ and $\hat{\mathbf{g}}_i$ are raw and normalized goal vectors, $\mathbf{p}_{\text{obs},c}$ is obstacle centroid, Π_i^{corr} and Π_i^{lat} are projections onto the inferred corridor and lateral axes, κ_t is formation scale, b_t is bottleneck score, π_t is normalized goal progress, ρ_i^{local} is a local crowding statistic, d_i^{min} is minimum obstacle distance, $\mathbf{v}_i^{\text{dyn}}$ is the nearest dynamic-obstacle velocity, χ_t is split activity, and m_t is the current topology mode. The dimensional decomposition is $2 + 2 + 2 + 2 + 2 + 2 + 2 + 2 + 2 + 2 + 3 + 2 + 2 + 1 + 5 + 1 + 36 = 68$.

Likewise, each directed edge ($j \rightarrow i$) carries

$$\mathbf{e}_{ij} = [\Delta \mathbf{p}_{ij}, \Delta \mathbf{v}_{ij}, \|\Delta \mathbf{p}_{ij}\|, \Pi_{ij}^{\text{corr}}, \Pi_{ij}^{\text{lat}}, \epsilon_{ij}^{\text{space}}, \text{TTC}_{ij}, b_t, \pi_t], \quad (7)$$

with dimension $2 + 2 + 1 + 1 + 1 + 1 + 1 + 1 + 1 = 11$. Both $\text{TTC}_i^{\text{min}}$ and TTC_{ij} are computed from the quadratic collision equation for circular agents rather than from a purely geometric distance threshold. The graph is directed because each robot independently selects its own six nearest neighbors.

The graph is processed by three attention-weighted message-passing layers. For edge ($j \rightarrow i$), an edge MLP produces a message $\mathbf{m}_{ij}$ and a learned attention weight α_{ij} , and the node update is

$$\mathbf{m}_{ij} = \phi_e([\mathbf{h}_i \parallel \mathbf{h}_j \parallel \mathbf{e}_{ij}]), \quad (8)$$

$$\alpha_{ij} = \text{softmax}_{j \in \mathcal{N}(i)}(\phi_a(\mathbf{m}_{ij})), \quad (9)$$

$$\mathbf{h}'_i = \mathbf{h}_i + \phi_n\left([\mathbf{h}_i \parallel \sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{m}_{ij}]\right). \quad (10)$$

The hidden dimension is 128 throughout. In the released model, an encoder MLP first maps $68 \rightarrow 128$, and each message-passing layer uses an edge MLP on $[\mathbf{h}_j \parallel \mathbf{h}_i \parallel \mathbf{e}_{ij}]$ followed by a per-destination softmax over incoming messages. Because k is fixed, the encoder cost scales approximately

linearly with swarm size, $\mathcal{O}(Nkd)$, which is one reason the policy can be evaluated on different team sizes without changing the architecture.

B. Recoverability Field Network

The shared backbone feeds four decoders:

- an **action head** producing per-robot accelerations $\mathbf{u}_i \in \mathbb{R}^2$;
- a **topology consensus head** producing graph-level logits over $\mathcal{T}$;
- a **score head** producing per-topology recoverability scores;
- an **uncertainty head** producing a nonnegative uncertainty estimate for each candidate topology.

The topology head is not a simple pooled classifier. Each node first casts a topology vote, neighbor votes are aggregated into mean and spread statistics, and a small agreement MLP refines these votes before graph-level pooling. This gives the model a limited form of local consensus without requiring explicit communication optimization. The consensus path is summarized in Algorithm 1.

Algorithm 1: Neighborhood Topology Consensus

Input: Node embeddings $\{\mathbf{h}_i\}$ and graph neighborhood $\mathcal{N}(i)$

Output: Graph-level topology logits ℓ

- 1 $\mathbf{v}_i \leftarrow \text{MLP}_{\text{vote}}(\mathbf{h}_i) \quad \forall i$;
 - 2 $\bar{\mathbf{v}}_i \leftarrow \text{MeanAgg}_{j \in \mathcal{N}(i)}(\mathbf{v}_j)$;
 - 3 $\sigma_i \leftarrow \sqrt{\text{VarAgg}_{j \in \mathcal{N}(i)}(\mathbf{v}_j) + \epsilon}$;
 - 4 $\hat{\mathbf{v}}_i \leftarrow \text{MLP}_{\text{agree}}([\mathbf{v}_i \parallel \bar{\mathbf{v}}_i \parallel \sigma_i])$;
 - 5 $\ell \leftarrow \text{MeanPool}(\hat{\mathbf{v}}_i)$;
 - 6 **return** ℓ
-

The four heads operate at two different granularities. Actions are robot-wise, whereas topology choice and recoverability are graph-wise. Let $\mathbf{h}_i^{\text{aux}} = \text{GradScale}(\mathbf{h}_i; 0.2)$ and $\bar{\mathbf{h}} = \frac{1}{N} \sum_i \mathbf{h}_i^{\text{aux}}$. The released implementation computes

$$\begin{aligned} \mathbf{u}_i &= \tanh(\phi_{\text{act}}(\mathbf{h}_i)), \\ \hat{\mathbf{r}} &= \phi_{\text{score}}(\bar{\mathbf{h}}), \\ \hat{\sigma} &= \text{softplus}(\phi_{\text{unc}}(\bar{\mathbf{h}})), \\ \hat{\mathbf{a}} &= \phi_{\text{aux}}(\bar{\mathbf{h}}), \end{aligned} \quad (11)$$

where the auxiliary head predicts four graph-level scalars: formation scale, bottleneck score, normalized progress, and split activity. This separation is deliberate: the continuous action is local and per-robot, while topology and recoverability are properties of the whole team configuration.

Recoverability scores are adjusted by uncertainty:

$$\tilde{r}_\tau = \hat{r}_\tau - 0.25 \hat{\sigma}_\tau, \quad \hat{R}(s_t) = \max_{\tau \in \mathcal{T}} \tilde{r}_\tau. \quad (12)$$

Only the action head receives full-strength gradients from the backbone. Before feeding the auxiliary heads, we apply gradient scaling with factor 0.2 to reduce degradation of control quality under multitask training. Concretely, the action head is a $128 \rightarrow 128 \rightarrow 64 \rightarrow 2$ MLP with tanh output, the topology vote head is $128 \rightarrow 64 \rightarrow |\mathcal{T}|$, the agreement MLP is $3|\mathcal{T}| \rightarrow 64 \rightarrow |\mathcal{T}|$, and the score / uncertainty /

auxiliary heads are $128 \rightarrow 128 \rightarrow |\mathcal{T}|$, $128 \rightarrow 128 \rightarrow |\mathcal{T}|$, and $128 \rightarrow 128 \rightarrow 4$, respectively.

C. Counterfactual Topology Selection

At inference, RVT-Swarm evaluates all candidate topologies using a combination of uncertainty-adjusted recoverability, topology prior, contextual bonuses, validity masks, and switching cost:

$$\begin{aligned} C_\tau &= 0.48 \pi_\tau + 0.30 \tanh(0.9 \tilde{r}_\tau) + b_\tau^{\text{ctx}} \\ &\quad - 0.06 \sigma_\tau - \psi_\tau^{\text{switch}} + m_\tau^{\text{valid}}, \end{aligned} \quad (13)$$

The context term favors, for example, LINE or COMPRESS in bottlenecks and RECOVER once the bottleneck clears. The switch penalty discourages excessive topology changes, especially disruptive split actions. The current selector is a two-stage gate rather than a single argmax: it starts from the topology logit winner, permits a recoverability-based override only when the score gain exceeds an uncertainty-dependent margin, and finally applies cooldown and hysteresis checks with additional margins for specialist actions (LINE, SPLIT, RECOVER). The online selection-and-shield loop is summarized in Algorithm 2. The exact released selector uses hard validity masks and coefficient-level penalties in addition to (13); we therefore list the concrete thresholds and margins in the appendix rather than pretending that the deployed implementation is a pure one-line score argmax.

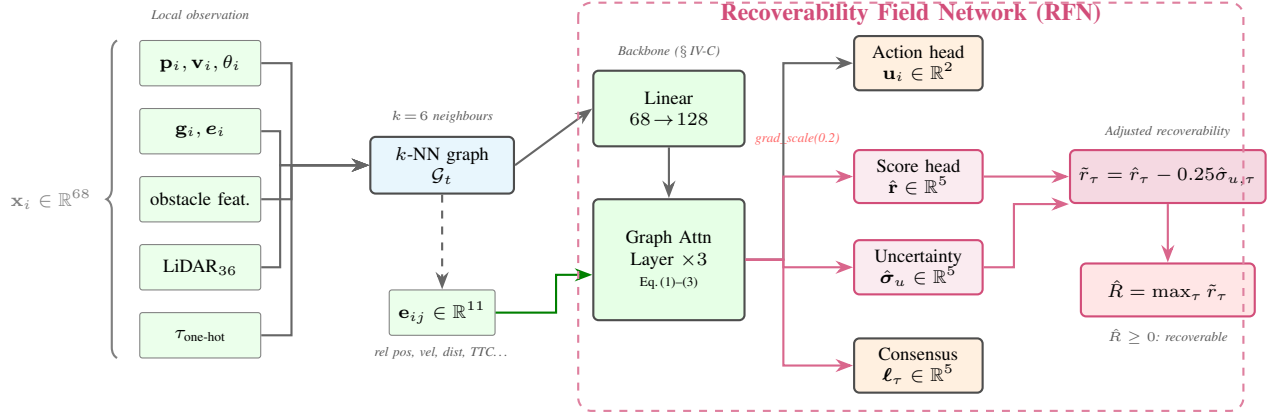


Fig. 2: Recoverability Field Network (RFN). Local node and edge features are encoded by a three-layer graph-attention backbone and decoded into action, recoverability, uncertainty, and topology outputs. The adjusted score map $\tilde{r}_\tau$ is used by the topology selector and by the shield trigger.

Algorithm 2: Counterfactual Topology Selection with Projection Shield

Input: Observation o_t , topology logits ℓ , adjusted scores $\tilde{r}$, uncertainty σ , previous topology τ_{prev} , nominal action $\mathbf{u}$

Output: Selected topology τ^* , shielded action $\mathbf{u}^{\text{safe}}$

```

1 if  $t - t_{\text{last-switch}} < T_{\text{cool}}$  then
2   |  $\tau^* \leftarrow \tau_{\text{prev}};$ 
3 end
4 else
5   | Compute prior  $\pi = \text{softmax}(\ell/T)$ ;
6   | Compute contextual bonuses  $b_\tau^{\text{ctx}};$ 
7   | Compute switching penalties  $\psi_\tau^{\text{switch}};$ 
8   | Compute  $C_\tau$  using (13);
9   |  $\tau' \leftarrow \arg \max_{\tau \in \mathcal{T}} C_\tau;$ 
10  | if  $C_{\tau'} < C_{\tau_{\text{prev}}} + \eta$  then
11    |  $\tau^* \leftarrow \tau_{\text{prev}};$ 
12  | else
13    |  $\tau^* \leftarrow \tau';$ 
14  | end
15 end
    // Risk-triggered projection shield
16  $\rho \leftarrow \text{CollisionRisk}(o_t);$ 
17 if  $\max_{\tau \in \mathcal{T}} \tilde{r}_\tau < 0$  then
18   |  $\rho \leftarrow \max(\rho, \rho_{\text{crisis}});$ 
19 end
20 if  $\rho \geq \rho_{\text{th}}$  then
21   | Build pairwise robot-robot and robot-obstacle half-planes;
22   | Project  $\mathbf{u}$  onto the feasible intersection and acceleration ball;
23   | Blend projected and nominal actions to obtain  $\mathbf{u}^{\text{safe}};$ 
24 end
25 else
26   |  $\mathbf{u}^{\text{safe}} \leftarrow \mathbf{u};$ 
27 end
28 return  $\tau^*, \mathbf{u}^{\text{safe}}$ 

```

D. Geometry Execution of Topology Actions

The selector does not directly reposition robots; instead it updates a latent formation state consisting of the discrete mode m_t , formation scale κ_t , split-activity variable χ_t , and optional subteam assignment vector $\mathbf{s}_t$. Desired offsets are then regenerated relative to the inferred corridor direction $\mathbf{c}_t$ and lateral axis $\mathbf{l}_t = [c_{t,y}, -c_{t,x}]^\top$.

For LINE, the desired geometry becomes a one-dimensional lattice along the corridor:

$$\delta_i^{\text{line}} = \mathbf{c}_t \left(i - \frac{N-1}{2} \right) d_0 \kappa_t. \quad (14)$$

For SPLIT, robots are sorted by lateral projection $\mathbf{p}_i^\top \mathbf{l}_t$ and divided into two subteams; each subteam occupies a lane centered at $\pm 0.55 g_t \mathbf{l}_t$ with longitudinal spacing $d_0 \kappa_t$, where $g_t = \max(0.7, 1.1 d_0 \kappa_t)$ is the lane gap. The nominal KEEP/COMPRESS/RECOVER modes use a grid aligned with $(\mathbf{c}_t, \mathbf{l}_t)$.

In the released simulator, topology actions act through smoothed state updates rather than abrupt geometry jumps: COMPRESS contracts κ_t toward a bottleneck-dependent target, LINE activates corridor alignment and a stronger contraction target, SPLIT activates two-lane assignments and increases χ_t , RECOVER clears split state and expands the formation, and KEEP either preserves the bottleneck configuration or drifts back toward the nominal grid. The topology-switch counter increments whenever the discrete mode changes or the scale changes by more than 0.06, which is why the selector must control both categorical switches and repeated scale-only oscillations.

E. Progress-Preserving Projection Shield

The shield activates only when collision risk is high or when all candidate topologies have negative recoverability. Pairwise robot-robot and robot-obstacle constraints are converted into half-plane constraints in action space using a CBF-inspired linearization. For robot i , we use the barrier

$$h_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\|^2 - d_{\text{safe}}^2 \quad (15)$$

and enforce

$$\nabla h_{ij}^\top \mathbf{u}_i \geq -\alpha h_{ij} - 2(\mathbf{p}_i - \mathbf{p}_j)^\top (\mathbf{v}_i - \mathbf{v}_j). \quad (16)$$

Rather than solving a full external QP, we iteratively project a progress-biased target action onto the intersection of these half-planes and the acceleration ball. The final shielded action is softly blended with the learned action, which reduces overreaction near the risk threshold. Recoverability is used conservatively: positive recoverability does not suppress geometric collision risk, but globally negative recoverability raises the intervention floor and therefore makes the shield more willing to intervene during unrecoverable crises. In the released artifact, collision risk itself is a max over proximity and TTC-based terms, the trigger threshold is additionally modulated by time since the last topology switch and current formation error, and the shield output is blended back into the learned action with a capped severity-dependent coefficient. These exact constants are included in the appendix for code-level reproducibility.

V. TRAINING

A. Data Generation

Training data are generated from a common heuristic expert controller in the shared simulator. For each sampled state, we run short-horizon counterfactual rollouts for every topology in $\mathcal{T}$ using the same expert controller, producing:

- a per-topology score vector;
- a best-topology label;
- a scalar recoverability margin derived from the score gap and the best score.

The current release uses 500 episodes spanning four scenarios (*open field*, *cluttered*, *narrow passage*, *dynamic obstacles*) and team sizes $N \in \{4, 8, 12, 16\}$. Scenario sampling probabilities are $[0.18, 0.28, 0.30, 0.24]$ for open, cluttered, narrow-passage, and dynamic-obstacle episodes, respectively. Hard-negative mining augments states with nonpositive recoverability labels and bottleneck score above 0.35 using perturbed actions.

For artifact fidelity, the shared expert itself is not an abstract oracle but a fixed controller used everywhere in the release for both label generation and counterfactual rollouts:

$$\mathbf{u}_i^{\text{expert}} = \text{clip}_{u_{\max}} \left(0.75 \hat{\mathbf{g}} + \gamma_\tau \mathbf{e}_i^\tau - 0.32 \mathbf{v}_i + \Delta_{i,\tau}^{\text{topo}} + \Delta_{i,\tau}^{\text{tr}} + \Delta_{i,\tau}^{\text{ro}} \right), \quad (17)$$

where $\hat{\mathbf{g}}$ is the centroid-to-goal unit vector, $\gamma_\tau \in \{0.98, 0.72, 0.48, 0.42, 0.98\}$ for $\{\text{KEEP}, \text{COMPRESS}, \text{LINE}, \text{SPLIT}, \text{RECOVER}\}$, and $\Delta_{i,\tau}^{\text{topo}}$ adds topology-specific terms such as corridor alignment for LINE, lane bias for SPLIT, and stronger error-correction for RECOVER. The terms $\Delta_{i,\tau}^{\text{tr}}$ and $\Delta_{i,\tau}^{\text{ro}}$ combine short-range repulsion with TTC-style urgency from other robots and obstacles. We write the exact piecewise controller in the appendix because the release really does depend on the precise coefficients.

The counterfactual rollout score that defines $\mathbf{q}(s_t)$ is also computed directly from the simulator rather than learned:

$$q_\tau(s_t) = \frac{1}{H} \left[\sum_{k=0}^{H-1} \left(0.95 c_k + 0.65 p_k + 1.55 T_k + 0.75 r_k^{\text{proxy}} + b_{k,\tau}^{\text{rec}} - d_k^{\text{lock}} - s_{k,\tau}^{\text{switch}} - \ell_{k,\tau}^{\text{split}} \right) + \sum_{k=0}^{H-1} 0.05 c_k + \beta_\tau^{\text{term}} \right], \quad (18)$$

where c_k is collision-free execution, p_k is normalized goal progress, T_k is the clipped formation-tube score, r_k^{proxy} is the simulator recoverability proxy, d_k^{lock} is a deadlock/stall penalty, $s_{k,\tau}^{\text{switch}}$ penalizes topology switching and specialist actions, $\ell_{k,\tau}^{\text{split}}$ penalizes lingering split states outside bottlenecks, and β_τ^{term} adds terminal success bonuses or an early collapse penalty. Again, the exact released coefficients are listed in the appendix rather than hidden inside “rollout score” shorthand.

Let $\mathbf{q}(s_t) \in \mathbb{R}^{|\mathcal{T}|}$ denote the rollout-score vector for a sampled state and let $\tau^* = \arg \max_{\tau \in \mathcal{T}} q_\tau$. The released code forms the supervised targets as

$$\begin{aligned} \mathbf{y}^{\text{score}} &= \frac{\mathbf{q} - \mu(\mathbf{q})\mathbf{1}}{\max(\sigma(\mathbf{q}), \epsilon)}, \\ y^{\text{topo}} &= \tau^*, \\ y^{\text{rec}} &= \tanh(0.55 [q_{\tau^*} - q_{(2)}] + 0.30 q_{\tau^*} + 0.18 \mathbb{I}[\tau^* = \text{RECOVER}]), \\ \mathbf{y}^{\text{act}} &= \mathbf{u}^{\text{expert}}(s_t, \tau^*) / u_{\max}, \end{aligned} \quad (19)$$

where $\mu(\mathbf{q})$ and $\sigma(\mathbf{q})$ are the mean and standard deviation of the rollout-score vector, $\mathbf{1}$ is the all-ones vector, ϵ is a small floor used to avoid division by a near-zero standard deviation, and $q_{(2)}$ is the second-best rollout score. Thus the scalar recoverability target is not an independent label; it is a distilled margin derived from the topology score vector itself. When $y^{\text{rec}} \leq 0$ and the bottleneck score exceeds 0.35, the dataset appends a hard negative by perturbing the expert action as $\tilde{\mathbf{u}} = 0.75 \mathbf{u}^{\text{expert}} + 0.25 \epsilon \mathbf{u}_{\max}$ with Gaussian noise ϵ , while keeping the same recoverability and topology labels.

B. Loss and Optimization

For RVT-Swarm, the training loss is

$$\mathcal{L} = \lambda_a \mathcal{L}_{\text{act}} + \lambda_r \mathcal{L}_{\text{rec}} + \lambda_\tau (\mathcal{L}_{\text{topo}} + 0.8 \mathcal{L}_{\text{score}} + 0.45 \mathcal{L}_{\text{rank}}) + \lambda_x \mathcal{L}_{\text{aux}} + \mathcal{L}_{\text{unc}}, \quad (20)$$

where the terms correspond respectively to action regression, recoverability regression, topology classification, score-map regression, pairwise topology ranking, auxiliary environment-state prediction, and uncertainty regularization. The current hyperparameters are $\lambda_a = 5.0$, $\lambda_r = 0.02$, $\lambda_\tau = 0.01$, and $\lambda_x = 0.004$. In the current release, the scalar recoverability regression term is retained for InstantCert but effectively disabled for topology-enabled RVT-Swarm (`rvt_recover_loss_scale=0.0`), because the score-map and ranking objectives were empirically more stable than jointly optimizing an additional scalar recoverability head. The ranking term is a unit-margin hinge loss over all topology

pairs with unequal target scores, and the uncertainty penalty is simply $0.02 \text{mean}(\hat{\sigma})$, discouraging gratuitous inflation of uncertainty without forcing the model to be overconfident.

Auxiliary losses are linearly ramped in over a 50-epoch warmup for RVT-Swarm. The three learned methods are trained with AdamW (3×10^{-4} learning rate, 10^{-5} weight decay, batch size 32). Best-checkpoint selection is based on periodic rollout validation: after warmup, every ten epochs we evaluate a small validation set over *narrow passage* and *dynamic obstacles* for team sizes 8 and 16 with four episodes per setting, and select checkpoints using

$$\begin{aligned} \text{Score}_{\text{val}} = & 1.00 \text{ Success} + 0.30 \text{ CollFree} + 0.20 \text{ FormOK} \\ & + 0.10 \text{ Goal} - 0.18 \text{ Collapse} - 0.08 \text{ Deadlock} \\ & - 0.03 \text{ Stall}. \end{aligned} \quad (21)$$

This is the exact scalar used by the training script once rollout-based checkpointing activates; it is not a hand-written summary added after the fact.

VI. EXPERIMENTS

A. Experimental Protocol

The benchmark uses a common 2D simulator with workspace 12×12 m, time step $\Delta t = 0.15$ s, robot radius 0.18 m, obstacle radius 0.35 m, speed limit 0.9 m/s, acceleration limit 0.6 m/s^2 , and a 36-ray LiDAR with 3 m range. Main evaluation uses 25 episodes per setting over 4 scenarios and 4 team sizes, for 400 episodes per method. Episodes are matched across methods by shared seeds,

$$\text{seed}(s, n, e) = s_0 + 10,000 \text{idx}(s) + 100n + e, \quad (22)$$

with $s_0 = 42$ in the current release; rollout-validation seeds use the same pattern with an additional offset of 50,000.

The primary success metric is also conjunctive in the released simulator:

$$\text{Success} = \mathbb{I}[\text{GoalReached} \wedge \text{CollisionFree} \wedge \text{FormOK}], \quad (23)$$

where *CollisionFree* requires zero robot-robot and robot-obstacle collisions over the episode, and *FormOK* = $\mathbb{I}[\text{FormRMS} < \epsilon_{\text{form}}]$. Because reviewers often ask whether these are post-processed metrics, we also give the exact released definitions of deadlock, irreversible collapse, and recoverability calibration in the appendix.

a) Implementation scope.: This benchmark is intentionally a *common-simulator prototype*. The non-learned baselines (ADAPTIVE_FORMATION, CBF_QP_LIKE, ORCA_LIKE, and CENTRALIZED_MPC) are internal normalized baselines rather than exact package-level reproductions. The current paper reports the single-seed result produced by the released code path; multi-seed confidence intervals remain future work. For artifact fidelity, we also note that the released `ms/step` field is a normalized runtime proxy from the evaluation script (1.0 for non-RVT methods and 1.6 for RVT-Swarm), not a wall-clock timing benchmark.

B. Compared Methods

GNN-Only uses the shared graph backbone with only an action head. **InstantCert** adds a scalar recoverability-style head but no topology adaptation. **RVT-Swarm** is the full model. Historical baselines include a heuristic adaptive-formation controller, a CBF-QP-like reactive filter, ORCA-like avoidance, and a centralized MPC-inspired proxy. For code-level reproducibility, these historical baselines are not left as labels:

- **ORCA-like** uses clipped goal tracking of the form $0.7 \hat{g} - 0.2 \mathbf{v}_i$ plus pairwise and obstacle repulsion, with no topology adaptation.
- **Adaptive formation** selects a heuristic topology from bottleneck/progress thresholds and then calls the same expert controller as in (17).
- **CBF-QP-like** starts from the KEEP-topology expert action and adds stronger reactive repulsion terms before clipping.
- **Centralized MPC proxy** also reuses the expert controller with a heuristic topology, then adds a small global centroid-to-goal bias.

These are therefore normalized internal baselines in one simulator, not claims of package-identical external reproductions.

C. Main Results

Three points stand out from Table II. First, the strongest learned baseline in the current artifact is InstantCert, which achieves the highest success, collision-free rate, formation satisfaction, and lowest collapse among learned methods. Second, RVT-Swarm nevertheless provides a distinct liveness profile: it has the lowest stall and deadlock rates among the learned controllers, suggesting that recoverability-aware topology control is currently more useful for avoiding stagnation than for maximizing raw completion rate. Third, the historical baselines still expose the intended trade-offs: ORCA-like motion reaches the goal often but abandons formation entirely, while CBF-QP-like control maximizes collision-free execution at the cost of formation completion.

For completeness, the released artifact also logs topology switches, formation recovery time, and recoverability calibration. In the latest run, RVT-Swarm averages 1.1425 topology switches and formation recovery time 204.67, compared with 0 switches and 201.91 for InstantCert. Recoverability false-positive / false-negative rates are 0.0565 / 0.0036 for RVT-Swarm versus 0.0451 / 0.0014 for InstantCert.

D. Ablation Results

TABLE III: Ablation results for RVT-Swarm. The current prototype ablation study is intentionally reported *as is*: the full model is not uniformly best on every metric.

Variant	Success $\uparrow$	Coll-Free $\uparrow$	Form OK $\uparrow$	Deadlock $\downarrow$	Collapse $\downarrow$	Topo Sw. $\downarrow$
Full	0.395	0.633	0.435	0.083	0.153	1.028
– Recoverability	0.380	0.610	0.433	0.083	0.130	0.793
– Counterfactual	0.350	0.620	0.398	0.083	0.160	2.542
– Progress shield	0.340	0.613	0.395	0.095	0.145	1.138
– Topology	0.453	0.665	0.495	0.108	0.150	0.000
Fixed formation	0.395	0.640	0.448	0.085	0.130	0.000

TABLE II: Main benchmark results averaged over all scenarios and team sizes. Numbers are from the current single-seed prototype run. The final column is a normalized runtime proxy from the released evaluation script, not a hardware-timed latency measurement.

Method	Success $\uparrow$	Goal $\uparrow$	Coll-Free $\uparrow$	Form OK $\uparrow$	Form RMS $\downarrow$	Stall $\downarrow$	Deadlock $\downarrow$	Collapse $\downarrow$	ms/step $\downarrow$
ORCA-like	0.000	0.920	0.633	0.000	2.907	0.005	0.010	0.018	1.0
Adaptive Formation	0.303	0.778	0.563	0.358	0.972	0.029	0.088	0.143	1.0
CBF-QP-like	0.183	0.753	0.723	0.205	0.847	0.027	0.080	0.085	1.0
Centralized MPC	0.318	0.763	0.603	0.345	0.963	0.030	0.105	0.163	1.0
GNN-Only	0.430	0.793	0.655	0.468	0.726	0.036	0.113	0.143	1.0
InstantCert	0.453	0.750	0.658	0.513	0.702	0.037	0.108	0.123	1.0
RVT-Swarm	0.425	0.785	0.640	0.490	0.777	0.028	0.083	0.128	1.6

The ablation study is the most important reason to keep the current paper honest. In the current artifact, the full model is not the strongest ablation variant on end-task success; instead, *no-topology* is best on success, collision-free execution, and formation satisfaction. At the same time, the ablation now shows that recoverability, counterfactual selection, and the progress shield all help relative to their direct removals: the full model outperforms —recoverability, —counterfactual, and —progress shield on success. Removing counterfactual selection is particularly damaging to topology stability, driving topology switches up to 2.54. Taken together, these results indicate that the recoverability and shield machinery are now useful, but the topology adaptation pathway itself remains under-tuned. We also emphasize an artifact-level caveat: in the released code, *fixed formation* and *no-topology* are nearly degenerate once topology use is disabled, so the gap between those two rows should not be over-interpreted under single-seed evaluation.

E. Interpretation

The current results suggest a more specific interpretation. Relative to InstantCert and GNN-Only, RVT-Swarm’s strongest contribution is lower liveness failure: it stalls less and deadlocks less, but does not yet translate that advantage into the highest overall success. The ablation table further indicates that the remaining gap is concentrated in the topology pathway rather than in the recoverability or shield modules: *no-topology* is the strongest ablation, while removing recoverability, counterfactual scoring, or the shield all makes the system worse. This is consistent with the codebase design: the recoverability labels are short-horizon counterfactual surrogates, and the selector still relies on hand-shaped context masks, switch penalties, and hysteresis margins rather than a jointly optimized planner.

VII. LIMITATIONS AND FUTURE WORK

This work should be read as a research prototype with promising algorithmic structure, not as a finished benchmark artifact. The main limitations are:

- **Short-horizon recoverability surrogate.** Recoverability labels are derived from a heuristic expert over a finite horizon rather than from a globally optimal planner or a formally certified reachability computation.
- **Prototype baselines.** The historical baselines are normalized in-house implementations in a common simulator, not exact reproductions of external packages.

- **Single-seed reporting.** The current paper reports one full prototype sweep. A submission intended for a top-tier venue should include multi-seed confidence intervals, statistical testing, and a more mature baseline suite.
- **Topology adaptation is not yet benchmark-positive.** The current ablation leader is the *no-topology* variant, indicating that the present selector/context dynamics still extract less end-task value than their complexity suggests.
- **Runtime proxy, not timing benchmark.** The released `ms/step` field is a normalized proxy baked into the evaluation script, not a synchronized wall-clock latency study.
- **Simplified shield and dynamics.** The shield uses iterative half-plane projection, and the environment is 2D with simplified kinematics. Extensions to nonholonomic and 3D platforms remain open.

The clearest next steps are to (i) align checkpointing and training losses more directly with rollout success, (ii) strengthen the recoverability label generation with better local planning or longer horizons, and (iii) evaluate the method over multiple seeds and richer baseline reproductions.

VIII. CONCLUSION

We presented RVT-Swarm, a decentralized graph-based controller that combines continuous control, discrete topology adaptation, and a learned recoverability score map for swarm formation control in constrained environments. The current implementation demonstrates that this design already reduces stall and deadlock relative to the simpler learned baselines, while also revealing through ablation that the main remaining gap lies in topology adaptation itself. In other words, the current artifact supports RVT-Swarm as a promising recoverability-aware formation-control scaffold, but not yet as a final benchmark-winning topology controller.

REFERENCES

- [1] K. Garg, S. Zhang, O. So, C. Dawson, and C. Fan, “Learning safe control for multi-robot systems: Methods, verification, and open challenges,” *Annual Reviews in Control*, vol. 57, p. 100948, 2024.
- [2] Z. Deng, P. Gao, W. J. Jose, and H. Zhang, “Multi-robot collaborative navigation with formation adaptation,” *arXiv preprint arXiv:2404.01618*, 2024.
- [3] Y. Xie, C. Yu, H. Zang, F. Gao, W. Tang, J. Huang, J. Chen, B. Xu, Y. Wu, and Y. Wang, “Multi-UAV formation control with static and dynamic obstacle avoidance via reinforcement learning,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 20410–20417.

- [4] B. Muthusamy, N. Mittal, Q. Li, A. Sahu, T. Chan, and A. Prorok, “Generalizability of graph neural networks for decentralized unlabeled motion planning,” *arXiv preprint arXiv:2404.16089*, 2024.

APPENDIX

APPENDIX: CURRENT HYPERPARAMETERS

TABLE IV: Current hyperparameter settings in the released prototype.

Parameter	Value
<i>Environment</i>	
Workspace	12.0 × 12.0 m
Δt	0.15 s
$v_{\max} / u_{\max}$	0.9 / 0.6
Robot / obstacle radius	0.18 / 0.35
LiDAR rays / range / FOV	36 / 3.0 m / 270°
Team sizes	{4, 8, 12, 16}
Scenarios	open, cluttered, narrow, dynamic
<i>Model</i>	
Node feature dim / edge dim	68 / 11
Hidden dim / message passes	128 / 3
Graph neighborhood k	6
Aux gradient scale	0.2
<i>Training</i>	
Expert episodes	500
Batch size	32
Learning rate / weight decay	$3 \times 10^{-4} / 10^{-5}$
Recoverability horizon H	14
λ_a / λ_r	5.0 / 0.02
λ_τ / λ_x	0.01 / 0.004
RVT warmup epochs	50
Early stopping patience	40
RVT max epochs	300
GNN / InstantCert max epochs	120 / 120
<i>Rollout validation</i>	
Enabled	yes
Interval	every 10 epochs
Scenarios	narrow passage, dynamic obstacles
Team sizes	8, 16
Episodes per setting	4
<i>Inference</i>	
Switch hysteresis	0.12
Topology cooldown	10 steps
Topology temperature	1.5
Shield risk threshold	0.85
Max shield blend	0.06
RVT scalar recover loss scale	0.0
<i>Evaluation</i>	
Episodes per setting	25
Runtime field	normalized proxy, not wall-clock

- [5] A. Goeckner, Y. Sui, N. Martinet, X. Li, and Q. Zhu, “Graph neural network-based multi-agent reinforcement learning for resilient distributed coordination of multi-robot systems,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.
- [6] S. Zhang, O. So, K. Garg, and C. Fan, “GCBF+: A neural graph control barrier function framework for distributed safe multi-agent control,” *IEEE Transactions on Robotics*, vol. 41, pp. 1533–1552, 2025.
- [7] B. A. Butler, C. H. Leung, and P. E. Paré, “Collaborative safety-critical formation control with obstacle avoidance,” *arXiv preprint arXiv:2410.03885*, 2024.
- [8] S. Zhang, O. So, M. Black, and C. Fan, “Discrete GCBF proximal policy optimization for multi-agent safe optimal control,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- [9] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n -body collision avoidance,” in *Robotics Research*. Springer, 2011, pp. 3–19.
- [10] A. Abate, S. Bogomolov, A. Edwards, K. Potomkin, S. Soudjani, and P. Zuliani, “Safe reach set computation via neural barrier certificates,” *IFAC-PapersOnLine*, vol. 58, no. 11, pp. 107–114, 2024.
- [11] G. Li, H. Mirinezhad, K. Garg, J. Sun, C. Fan, B. Ghahesifard, A. A. Ahmadi, and A. K. Sojoudi, “Certifiable reachability learning using a new lipschitz continuous value function,” *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3582–3589, 2025.
- [12] Y. Jung, M. Kim, H. J. Lim, and M. Pavone, “RAIL: Reachability-aided imitation learning for safe policy execution,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 4903–4909.

APPENDIX: ARTIFACT-FAITHFUL IMPLEMENTATION DETAILS

- All learned methods are trained on the same generated dataset. For the current 500-episode seed used in the released run, the shared dataset contains 58,763 graph samples.
- The simplified learned baselines are also fully specified in code: GNN-Only uses a shared backbone plus an action head $128 \rightarrow 128 \rightarrow 2$, while InstantCert uses the same action head plus a pooled scalar-certification head $128 \rightarrow 128 \rightarrow 1$.
- The exact released decoder widths are: encoder $68 \rightarrow 128 \rightarrow 128$, action head $128 \rightarrow 128 \rightarrow 64 \rightarrow 2$, topology vote head $128 \rightarrow 64 \rightarrow 5$, agreement head $15 \rightarrow 64 \rightarrow 5$, score head $128 \rightarrow 128 \rightarrow 5$, uncertainty head $128 \rightarrow 128 \rightarrow 5$, and auxiliary head $128 \rightarrow 128 \rightarrow 4$.
- The scalar recoverability target is distilled from the rollout-score vector rather than annotated separately: the code z-normalizes the rollout scores, uses the argmax as the topology label, and maps the best-score gap to a scalar margin through the tanh expression in (19). Action targets are normalized by the maximum acceleration so that the tanh-bounded action head predicts in $[-1, 1]^2$.
- The exact expert controller used for both dataset generation and counterfactual rollouts is

$$\mathbf{u}_i^{\text{expert}} = \text{clip}_{u_{\max}} \left(0.75 \hat{\mathbf{g}} + \gamma_\tau \mathbf{e}_i^\tau - 0.32 \mathbf{v}_i + \Delta_{i,\tau}^{\text{topo}} + \sum_{j \neq i} \Delta_{ij}^{\text{rr}} + \sum_k \Delta_{ik}^{\text{ro}} \right),$$

with $\gamma_\tau = 0.98$ for KEEP/RECOVER, 0.72 for COMPRESS, 0.48 for LINE, and 0.42 for SPLIT. The topology-specific residual is

$$\Delta_{i,\tau}^{\text{topo}} = \begin{cases} 0.06 \hat{\mathbf{g}}, & \tau = \text{COMPRESS}, \\ 0.15 (\mathbf{e}_i^\top \mathbf{l}^\top \mathbf{c}), & \tau = \text{LINE}, \\ 0.18 \sigma_i \mathbf{l} + 0.08 \hat{\mathbf{g}}, & \tau = \text{SPLIT}, \\ 0.22 \mathbf{e}_i^\top - 0.10 \mathbf{v}_i + 0.05 \hat{\mathbf{g}}, & \tau = \text{RECOVER}, \\ 0, & \tau = \text{KEEP}, \end{cases}$$

where $\mathbf{c}$ is the corridor axis, $\mathbf{l}$ its lateral axis, and $\sigma_i \in \{-1, +1\}$ is the split lane sign. The short-range / TTC robot term is

$$\Delta_{ij}^{\text{rr}} = 0.32 [1 - d_{ij}]_+ \hat{\mathbf{n}}_{ij} \mathbb{I}[d_{ij} < 1.0] + 0.35 \left[1 - \frac{\text{TTC}_{ij}}{2.5}\right]_+ \hat{\mathbf{n}}_{ij} \mathbb{I}[d_{ij} < 2.5, \dot{d}_{ij} < -0.05],$$

and the obstacle term is

$$\Delta_{ik}^{\text{ro}} = 0.9 [1.3 - d_{ik}]_+ \hat{\mathbf{n}}_{ik} \mathbb{I}[d_{ik} < 1.3] + 0.5 \left[1 - \frac{\text{TTC}_{ik}}{2.5}\right]_+ \hat{\mathbf{n}}_{ik} \mathbb{I}[d_{ik} < 2.5, \dot{d}_{ik} < -0.05].$$

- The exact rollout score used to form counterfactual labels is

$$q_\tau = \frac{1}{H} \left[\sum_{k=0}^{H-1} \left(0.95 c_k + 0.65 p_k + 1.55 T_k + 0.75 r_k^{\text{proxy}} + b_{k,\tau}^{\text{rec}} - d_k^{\text{lock}} - s_{k,\tau}^{\text{switch}} - \ell_{k,\tau}^{\text{split}} \right) + \sum_{k=0}^{H-1} 0.05 c_k + \beta_\tau^{\text{term}} \right],$$

where $T_k = \text{clip}(1 - \text{FormRMS}_k / (1.45 \epsilon_{\text{form}}), 0, 1)$,

$$d_k^{\text{lock}} = 0.7 \text{Deadlock}_k + 0.5 \mathbb{I}[\text{StallRate}_k > 0.35] + 0.2 \text{StallRate}_k,$$

$$s_{k,\tau}^{\text{switch}} = 0.03 n_k^{\text{switch}} + 0.06 \mathbb{I}[n_k^{\text{switch}} > 1] + 0.03 \mathbb{I}[\tau \in \{\text{LINE}, \text{SPLIT}, \text{RECOVER}\}] + 0.06 \mathbb{I}[\tau \in \{\text{LINE}, \text{SPLIT}\}, b_k < 0.48] + 0.06 \mathbb{I}[\tau = \text{RECOVER}, b_k > 0.38, T_k > 0.72],$$

$$b_{k,\tau}^{\text{rec}} = 0.14 T_k \mathbb{I}[\tau = \text{RECOVER}] + 0.06 \mathbb{I}[\tau = \text{RECOVER}, \chi_k > 0] + 0.10 \mathbb{I}[\tau = \text{RECOVER}, b_k < 0.28] + 0.03 \mathbb{I}[\tau \in \{\text{LINE}, \text{SPLIT}\}, b_k > 0.52],$$

$$\ell_{k,\tau}^{\text{split}} = 0.16 \mathbb{I}[\chi_k > 0.45, b_k < 0.25].$$

$$\beta_\tau^{\text{term}} = 1.2 \text{Success} + 0.3 \text{GoalReached} - 1.0 \mathbb{I}[k \geq 3, \text{Collapse} > 0.5].$$

- Dynamic obstacles reflect at workspace boundaries, and robot-robot / robot-obstacle overlaps are resolved by iterative position projection rather than by impulse-based rigid-body simulation.
- The exact topology-state updates used by the released simulator are coefficient-level rules, not abstract mode toggles: for example, COMPRESS moves the scale toward $1 - 0.38 \max(b_t, 0.3)$, LINE moves it toward 0.52 or 0.72 depending on bottleneck severity, SPLIT moves it toward 0.8 while increasing split activity by 0.22, RECOVER expands by 0.22 or 0.12 depending on bottleneck score and clears the split state, and KEEP drifts toward 0.82 or 1.0 depending on whether the team is still inside a bottleneck.
- The released topology selector is more concrete than the compact presentation in (13). It uses $\text{softmax}(\ell/1.5)$ as the prior, validity masks requiring bottleneck ≥ 0.44 for LINE, bottleneck ≥ 0.60 and $N \geq 12$ for SPLIT, and RECOVER disabled only when progress < 0.65 , split activity < 0.28 , and formation error is already below tolerance. The exact switch penalty is

$$\psi_\tau^{\text{switch}} = (0.08 + 0.08 \rho_{\text{cool}}) \mathbb{I}[\tau \neq \tau_{\text{prev}}] + 0.04 \mathbb{I}[\tau_{\text{prev}} = \text{KEEP}, \tau \in \{\text{LINE}, \text{SPLIT}, \text{RECOVER}\}] + 0.16 \mathbb{I}[\tau = \text{SPLIT}] + 0.07 \mathbb{I}[\tau = \text{RECOVER}, b_t > 0.40] + 0.06 \mathbb{I}[\tau = \text{KEEP}, b_t > 0.55],$$

with cooldown fraction and margins

$$\rho_{\text{cool}} = \text{clip}\left(\frac{T_{\text{cool}} - \Delta t_{\text{switch}}}{T_{\text{cool}}}, 0, 1\right), \quad T_{\text{cool}} = 10,$$

$$m_{\text{override}} = 0.24 + 0.08[\bar{\sigma} - 0.35]_+ + 0.05 \mathbb{I}[\tau \in \{\text{LINE}, \text{SPLIT}, \text{RECOVER}\}] + 0.04 \mathbb{I}[\tau_{\text{prev}} = \text{KEEP}, \tau \in \{\text{LINE}, \text{SPLIT}, \text{RECOVER}\}],$$

$$m_{\text{hyst}} = 0.12 + 0.04 \mathbb{I}[\tau \in \{\text{LINE}, \text{SPLIT}, \text{RECOVER}\}] + 0.03 \mathbb{I}[\tau_{\text{prev}} = \text{KEEP}, \tau \in \{\text{LINE}, \text{SPLIT}, \text{RECOVER}\}] + 0.06[\bar{\sigma} - 0.35]_+.$$

- The released shield computes collision risk as the maximum of proximity and TTC-based terms:

$$\text{Risk} = \max\left([0.9 - d_{ij}]_+, 0.45[1 - \text{TTC}_{ij}/2]_+, [0.95 - d_{ik}]_+, 0.45[1 - \text{TTC}_{ik}/2]_+\right),$$

raises it when scalar recoverability is < -0.10 or when all per-topology recoverability scores are negative, activates when the risk exceeds a base threshold

of 0.85 (plus 0.05 if the last switch was within four steps and plus 0.03 for specialist topologies under poor formation), solves per-robot half-plane projections with progress weight 0.008 or 0.02 in full crisis, and blends the projected action back with a cap of 0.06.

- The main evaluation metrics are exactly those produced by the simulator:

$$\text{CollisionFree} = \mathbb{I}[\text{RRCol} = 0 \wedge \text{ROCol} = 0],$$

$$\text{FormOK} = \mathbb{I}[\text{FormRMS} < \epsilon_{\text{form}}],$$

$$\text{Deadlock} = \mathbb{I}[\text{stall_counter} \geq 12 \wedge \neg \text{GoalReached}],$$

$$\text{Collapse} = \mathbb{I}[(\text{RRCol} + \text{ROCol} > 0.05 \wedge \text{FormRMS} > 1.8 \epsilon_{\text{form}}) \vee \text{Deadlock} \vee (\chi_t > 0.55 \wedge b_t < 0.25 \wedge \text{FormRMS} > 1.2 \epsilon_{\text{form}})],$$

while recoverability false positives / false negatives are per-step rates for predicting safe during collapse or predicting unsafe when collapse does not occur.

- The evaluation artifact stores additional metrics beyond the main paper tables, including topology switches, formation recovery time, and recoverability false-positive / false-negative rates.
- The experiment driver runs the full artifact in the order: shared learned-method dataset generation, training of {GNN_ONLY, INSTANT_CERT, RVT_SWARM}, evaluation of those three learned methods plus {ADAPTIVE_FORMATION, CBF_QP_LIKE, ORCA_LIKE, CENTRALIZED_MPC}, shared ablation dataset generation, six ablation train/eval runs, and optional GIF visualization.
- The ablation toggles are exactly: full, no_recoverability, no_counterfactual, no_progress_shield, no_topology, and fixed_formation; the last two are nearly degenerate once topology execution is disabled, which is why their gap should not be over-interpreted under single-seed runs.
- The released visualization pipeline also renders per-method and comparison GIFs for *open field* and *narrow passage* scenarios at team sizes 4 and 8, which is useful for qualitative diagnosis of topology switching behavior.